



Tecnicatura Universitaria
en Programación

PROGRAMACIÓN IV

BACK END

Anexo:

Clientes HTTP – Docker Distroless

Anexo
2° Año – 4° Cuatrimestre



Cientes HTTP en Java 2

¿Por qué hablar de “clientes”?	2
Definición operativa	2
Responsabilidades principales	3
Resiliencia y observabilidad desde el cliente	3
Patrones relacionados	5
Malas Practicas	5

Programación Reactiva con Spring WebFlux 6

¿Qué es?	6
WebFlux	6
Operadores	7
Operadores de transformación	8
Operadores de filtrado y combinación	8
Operadores de control y resiliencia	8
Operadores de acumulación y cierre	8
Tabla visual de los operadores	9
Patrón de Arquitectura Relacionado	9
Caso ilustrativo con colas (RabbitMQ)	9
Tecnologías para Programación Reactiva	10
Redis – Mensajería ligera y cache reactiva	10
WebSocket – Comunicación en tiempo real con el cliente	11
RabbitMQ – Comunicación por colas de mensajes	11

Docker Distroless 12

Multi-stage builds	12
Dockerfile de referencia	12
Diferencias entre Dockerfile “normal” y Dockerfile “Distroless”	13

Cientes HTTP en Java

¿Por qué hablar de “clientes”?

En microservicios, un servicio le pide datos o acciones a otro usando HTTP. El “cliente” es el componente que inicia esa solicitud. Pensalo como la persona que hace un llamado telefónico: marca el número (URL), dice qué necesita (método + body), y escucha la respuesta (status + JSON).

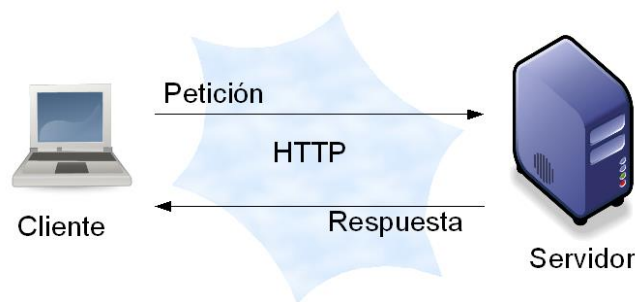


Imagen 1: Elaboración propia.

Definición operativa

Un cliente HTTP es un ensamble mediante el cual delegás la responsabilidad a un componente de adaptarse a la respuesta de otro servidor. Se encarga de:

- Construir la request (URL, query params, headers, serialización JSON).
- Manejar la respuesta y traducir errores (4xx/5xx) a excepciones de negocio.
- Aislar detalles técnicos (timeouts, reintentos, autenticación) para que el resto del código no los repita.

Idea clave: el dominio (reglas de negocio y modelos de la aplicación, independientes de frameworks) define “qué” necesitas; el cliente resuelve el “cómo” se llama al servicio externo.

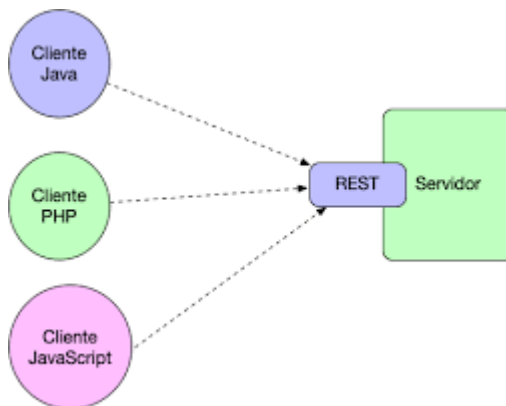


Imagen 2: Elaboración propia.

Responsabilidades principales

Cómo se comporta ante las respuestas: Cuando el servicio responde 2xx, el cliente deserializa tolerando campos extra, valida lo mínimo y mapea al modelo del dominio (opcionalmente cachea). Ante 3xx sigue o rechaza redirecciones según la política definida.

Si llega un 4xx, no reintenta: traduce el error remoto a uno de negocio (p. ej., 400/422 datos inválidos; 401/403 credenciales/autorización; 404 no encontrado; 409 conflicto). Frente a 429 respeta Retry-After y aplica backoff. Con 5xx o timeouts, reintenta de forma acotada y, si persiste, abre el circuito y devuelve un fallback o informa indisponibilidad.

Para 202/204 maneja asincronía o ausencia de body sin fallar; en respuestas paginadas sigue el token/next cuando corresponda; y siempre registra status, latencia y trazas.

Se podría clasificar las responsabilidades dentro de estos grupos:

- Construcción: URL y headers claros, serialización/deserialización confiable.
- Gestión de errores: mapear 404/409/500 a errores que *entienda* el negocio.
- Políticas: timeouts, retries con backoff, circuit breaker, rate limits.
- Configuración: endpoints por ambiente, *service discovery*, versionado de contratos.

Resiliencia y observabilidad desde el cliente

Cuando hablamos de resiliencia, nos referimos a la capacidad del cliente para *seguir funcionando correctamente* incluso si los servicios a los que llama presentan fallas, demoras o están temporalmente fuera de línea. Imaginá que hacés un pedido a un restaurante por teléfono: si la línea está ocupada, cortás y volvés a intentar después de unos minutos; si no atienden varias veces, asumís que están cerrados y buscás otra opción. El cliente HTTP hace algo parecido: define tiempos máximos de espera (timeouts), reintentos limitados y, si detecta fallas repetidas, “corta el circuito” temporalmente (patrón *Circuit Breaker*) para evitar sobrecargar un servicio que ya está en problemas.

La observabilidad, en cambio, trata de poder *entender qué pasó* con cada llamada.

Cada vez que el cliente se comunica con otro servicio, deja huellas: registra el método y la URL usados, cuánto tardó la respuesta, qué código de estado devolvió y un identificador de trazado (*traceId*) que permite seguir todo el recorrido de una solicitud entre microservicios.

A partir de ahí:

- Los logs cuentan la historia paso a paso.
- Las métricas muestran el rendimiento a lo largo del tiempo (por ejemplo, la tasa de éxito o la latencia promedio).
- Las trazas ayudan a reconstruir visualmente el camino que siguió una petición cuando algo falla o se vuelve lento.

Podés imaginarlo como un sistema de cámaras y sensores en una autopista: la resiliencia asegura que el tráfico siga fluyendo incluso si un carril se bloquea; la observabilidad permite entender dónde ocurrió el problema y por qué.

Ejemplo (cambio de firma → 400 Bad Request)

- Ayer `/users/auth/register` aceptaba:

```
{
  "email": "ana@mail.com",
  "password": "Secreta#1"
}
```

- Hoy el servicio cambió la firma y exige un campo nuevo (`country`) y valida longitud mínima de password:

```
{
  "email": "ana@mail.com",
  "password": "Secreta#1",
  "country": "AR"
}
```

- Nuestro cliente envía el **payload** viejo y recibe **400 Bad Request** con un cuerpo de error:

```
{
  "error": "BAD_REQUEST",
  "details": [
    {
      "field": "country",
      "message": "is required"
    },
    {
      "field": "password",
      "message": "min length 12"
    }
  ]
}
```

Patrones relacionados

Un problema común que podríamos tener Cuando un servicio remoto está inestable (errores 5xx repetidos, timeouts, latencias altas) y las llamadas continuas empeoran todo (efecto dominó), el cliente necesita cortar temporalmente para evitar saturar al sistema y degradar la experiencia. Aplicaríamos **Circuit Breaker** como el disyuntor eléctrico, cortando las llamadas si detecta fallas repetidas.

Otro problema al momento de tener varios microservicios es si una operación de negocio cruza varios servicios y no hay transacción distribuida. Si un paso intermedio falla, necesitamos compensar pasos previos para no dejar el sistema en estado inconsistente. Podríamos aplicar **SAGA** coreografías de pasos con *compensaciones* cuando una transacción larga debe deshacerse parcialmente.

Malas Practicas

El error más frecuente es **llamar directamente al controller** de otro servicio. Eso rompe el diseño: el *controller* está pensado para el mundo **externo**, no para llamadas internas. Entre microservicios, la comunicación debe pasar siempre por una **capa Client**, ubicada dentro del Service. Así mantenés un flujo ordenado, trazable y fácil de probar. Otros errores comunes son:

- No manejar errores ni timeouts: Un cliente que no define tiempos de espera puede quedar bloqueado si el servicio remoto tarda o no responde. Y si además no maneja errores HTTP (4xx o 5xx), el sistema se vuelve una caja negra: no sabés por qué falló.
- Reintentos infinitos o sin control: Cuando el otro servicio falla, algunos clientes reintentan automáticamente... una, dos, cien veces. Eso solo agrava el problema: saturás la red y generás una tormenta de peticiones.
- No desacoplar contratos: Copiar o importar los DTOs del otro microservicio parece rápido, pero crea dependencia. Si el otro cambia un campo, tu cliente deja de funcionar sin previo aviso.
- Falta de observabilidad: Un cliente sin logs ni métricas es un cliente ciego.
No saber qué endpoint llamó, cuánto tardó o qué status recibió te deja sin herramientas para diagnosticar.

Programación Reactiva con Spring WebFlux

¿Qué es?

Trabajamos con flujos de datos que llegan con el tiempo. En lugar de “esperar bloqueado”, nos suscribimos y reaccionamos cuando aparecen eventos. En un ejemplo cotidiano podríamos analizar las aplicaciones de deliveries:

1. *Pedido aceptado* → te avisan solo cuando pasa (no preguntas cada minuto).
2. *En preparación* → llega otro evento.
3. *En camino* → otra actualización.
4. *Llegando* → alerta final.

No hacemos “polling” constante; recibimos eventos. Podemos relacionarlo al cliente HttpClient de angular. Es decir que al momento de hacer una llamada nuestro programa libera un hilo aparte el cual espera el evento de la respuesta y nuestro servicio pueda seguir con el hilo principal.

WebFlux

Para implementar WebFlux tenemos que tener en cuenta que a nivel mental: **Mono** es como un **Optional<T>** **asíncrono** (0 o 1 valor), **Flux** es como un **List<T>** **asíncrono** (0, N valores). “Asíncrono” significa que no bloquea el hilo mientras espera resultados; el dato llega después y te suscribís para reaccionar cuando aparezca.

A continuación, veremos en controlador que emite resultados de forma reactiva:

```
@RestController
@RequestMapping("/api")
public class DemoController {

    // 0..1 resultado
    @GetMapping("/hola")
    public Mono<String> hola() {
        return Mono.just( data: "Hola WebFlux!");
    }

    // 0..N resultados
    @GetMapping(value = "/numeros", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
    public Flux<Integer> numeros() {
        return Flux.range( start: 1, count: 5); // 1,2,3,4,5
    }
}
```

Imagen 3: Elaboración propia.

- **Mono.just("Hola WebFlux!")**: crea un flujo que emitirá un String y completará.
- **Flux.range(1,5)**: crea un flujo que emitirá cinco enteros.
- **TEXT_EVENT_STREAM** te permite “streamear” valores uno a uno (útil para SSE).

Por ejemplo, a continuación, se muestra un cliente que consume otro servicio mediante **WebClient**, la implementación reactiva de **WebFlux** para consumir servicios.

```
@Service no usages
@RequiredArgsConstructor
class UserClient {
    private final WebClient webClient;

    public Mono<UserDto> getUserById(String id) { no usages
        return webClient.get() RequestHeadersUriSpec<capture of ?>
            .uri( uri: "/users/{id}", id) capture of ?
            .retrieve() ResponseSpec
            .bodyToMono(UserDto.class); // Mono<UserDto>
    }

    public Flux<UserDto> getAllUsers() { no usages
        return webClient.get() RequestHeadersUriSpec<capture of ?>
            .uri( uri: "/users") capture of ?
            .retrieve() ResponseSpec
            .bodyToFlux(UserDto.class); // Flux<UserDto>
    }
}
```

Imagen 4: Elaboración propia.

Operadores

En WebFlux, los datos no llegan todos juntos, sino que fluyen en el tiempo. Cada flujo puede representar un solo valor (**Mono**) o muchos valores (**Flux**). La magia de esta forma de programación está en los operadores, pequeñas funciones que nos permiten transformar, combinar o controlar ese flujo de datos sin bloquear el sistema.

Operadores de transformación

Estos operadores cambian el contenido del flujo, ya sea modificando los datos o encadenando nuevas llamadas.

- **map**: transforma cada elemento de manera directa (como aplicar **toUpperCase** sobre un string).
- **flatMap**: transforma cada elemento, pero devuelve otro flujo asíncrono (útil cuando una operación depende de otra).

Operadores de filtrado y combinación

Permiten elegir qué elementos pasan y fusionar flujos distintos.

- **filter**: deja pasar solo los elementos que cumplan una condición.
- **zip**: combina varios flujos y los emite juntos, como si hiciera un “**join**” reactivo.

Operadores de control y resiliencia

Estos operadores hacen que el cliente sea resiliente, capaz de reaccionar a fallos sin bloquear o romper la aplicación. Es mejor fallar rápido y controlado que colgar todo el sistema esperando una respuesta que nunca llega.

- **timeout**: corta si el servicio tarda más de lo esperado.
- **retry**: reintenta llamadas fallidas (solo si vale la pena).
- **onErrorResume**: captura errores y devuelve una alternativa.

Operadores de acumulación y cierre

Cuando necesitas transformar un Flux (muchos elementos) en algo único al final, usas operadores que “**recogen**” los datos.

- **collectList()**: acumula todos los elementos en una **List**.
- **reduce()**: aplica una operación acumulativa (como sumar números o combinar objetos).

Tabla visual de los operadores

Tipo de operador	Qué hace	Ejemplo típico
Transformación	Modifica o encadena datos	<code>map, flatMap</code>
Filtrado/Combinación	Selecciona o mezcla flujos	<code>filter, zip</code>
Control/Resiliencia	Maneja tiempos y errores	<code>timeout, retry, onErrorResume</code>
Acumulación/Cierre	Reúne datos finales	<code>collectList, reduce</code>

Tabla 1: Elaboración propia.

Patrón de Arquitectura Relacionado

Event-Driven Architecture (EDA)

En este patrón, los servicios se comunican por eventos en lugar de llamadas directas. Por ejemplo, un servicio publica un hecho (“pedido confirmado”) y otros reaccionan sin depender del primero. Esto hace que el sistema sea más desacoplado, escalable y resiliente.

Streaming

El streaming se usa cuando los datos fluyen de manera continua en lugar de llegar todos juntos. WebFlux permite emitir resultados parciales en tiempo real, usando Flux.

Caso ilustrativo con colas (RabbitMQ)

Imaginemos un sistema donde el frontend registra un nuevo usuario y queremos que, al mismo tiempo, otro servicio le envíe un correo de bienvenida. En lugar de hacer una llamada directa (sincrónica) entre servicios, usamos un flujo de eventos apoyado en RabbitMQ.

1. El frontend dispara “**registrar usuario**”. El cliente (por ejemplo, una app Angular) envía una petición **POST /users/register** al **backend**. Este endpoint recibe los datos del nuevo usuario y los valida.

2. El servicio de usuarios publica un mensaje en **RabbitMQ**. Una vez registrado el usuario en la base de datos, el servicio crea un evento con la información relevante (ID y correo) y lo publica en **RabbitMQ**.
3. **RabbitMQ** encola el mensaje. Recibe el evento y lo guarda en una cola (queue) llamada, por ejemplo, **user.notifications**. Esta cola actúa como un “buzón”: si el consumidor está caído o ocupado, el mensaje espera hasta que pueda ser procesado.
4. El servicio de notificaciones consume reactivamente. Otro microservicio (por ejemplo, **NotificationService**) está suscripto a la cola. Cuando llega un mensaje, lo consume reactivamente y ejecuta la acción correspondiente.

En **WebFlux**, esta lógica puede integrarse con un Flux que escuche continuamente los eventos, manteniendo el flujo no bloqueante.

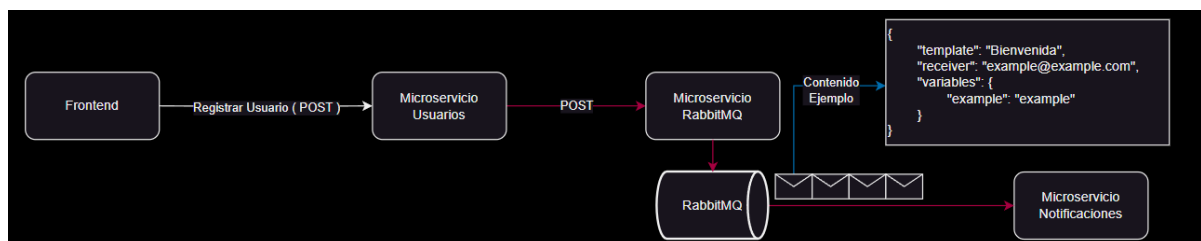


Imagen 5: Elaboración propia.

Tecnologías para Programación Reactiva

Redis – Mensajería ligera y cache reactiva

Redis es una base de datos en memoria, extremadamente rápida, que puede usarse tanto para cachear datos como para comunicar eventos entre servicios.

Cómo ayuda a lo reactivo:

- Permite publicar y suscribirse a canales (pub/sub) sin bloquear hilos.
- Puede actuar como broker de eventos liviano, ideal para sistemas que necesitan compartir mensajes cortos o notificaciones instantáneas.
- También se usa como cache reactiva, evitando llamadas innecesarias a servicios externos.

WebSocket – Comunicación en tiempo real con el cliente

WebSocket es un canal de comunicación bidireccional y persistente entre el servidor y el cliente (por ejemplo, el navegador). A diferencia del HTTP clásico (que cierra la conexión después de cada request), WebSocket mantiene la conexión abierta, permitiendo enviar y recibir datos en tiempo real.

Cómo ayuda a lo reactivo:

- Ideal para apps con actualización instantánea (chats, notificaciones, dashboards).
- Se integra fácilmente con Flux, emitiendo eventos que el cliente recibe sin necesidad de “refrescar”.

RabbitMQ – Comunicación por colas de mensajes

RabbitMQ es un message broker: un sistema que recibe, guarda y distribuye mensajes entre microservicios. Los mensajes se encolan y cada servicio los consume a su propio ritmo, sin bloquear a los demás.

Cómo ayuda a lo reactivo:

- Permite que los servicios se comuniquen sin esperarse mutuamente.
- Si un servicio está caído, los mensajes se guardan hasta que vuelva.
- Favorece la tolerancia a fallos y la escalabilidad del sistema.

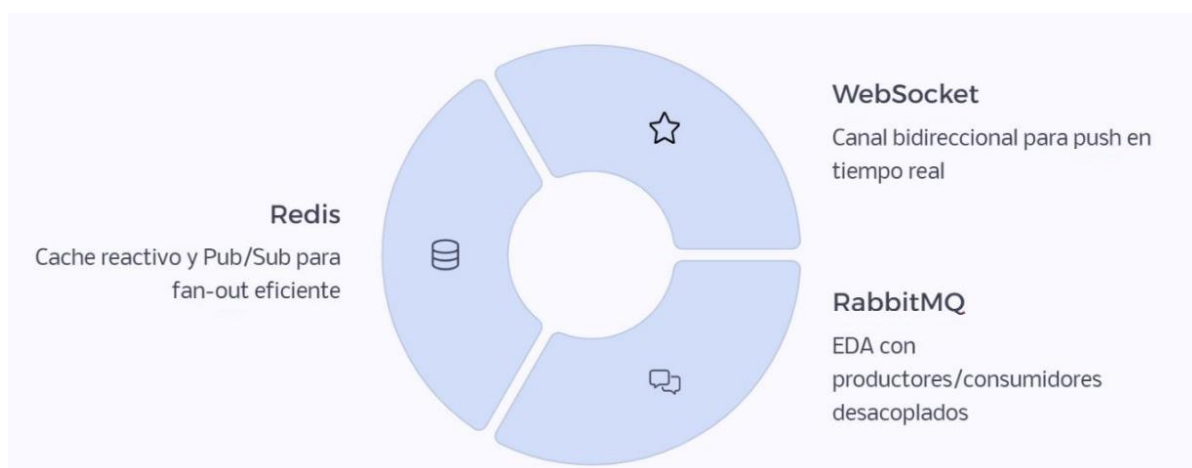


Imagen 6: Elaboración propia.

Docker Distroless

Una imagen **Distroless** es un tipo especial de **imagen Docker** que **no contiene un sistema operativo completo**, sino solo lo necesario para ejecutar tu aplicación (por ejemplo, el *runtime* de Java o Node.js).

A diferencia de las imágenes tradicionales (como Ubuntu, Debian o Alpine), no trae bash, curl ni herramientas de depuración. Esto la hace más liviana, más segura y más predecible en entornos de producción.

Podés pensarla como una caja sellada y optimizada: solo tiene lo justo para correr tu app, nada más.

Multi-stage builds

Para llegar a una imagen Distroless se usa una técnica llamada multi-stage build, donde el Dockerfile se divide en etapas separadas:

- **Stage 1 – Build:** Se usa una imagen completa (por ejemplo, con Maven o Gradle) para compilar y empaquetar la aplicación. Esta etapa tiene todas las herramientas necesarias para construir el .jar, pero no será parte de la imagen final.
- **Stage 2 – Runtime:** Solo se copia el artefacto final (el .jar) desde el primer stage hacia una imagen Distroless. El resultado es una imagen extremadamente liviana y lista para producción.

Dockerfile de referencia

```
FROM openjdk:17-slim AS build
RUN apt-get update && apt-get install -y maven && apt-get clean
WORKDIR /app
COPY . /app
RUN mvn clean package -DskipTests
FROM gcr.io/distroless/java17
WORKDIR /app
COPY --from=build /app/target/*.jar /app/app.jar
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

Imagen 7: Elaboración propia.

Diferencias entre Dockerfile “normal” y Dockerfile “Distroless”

Característica	Dockerfile normal	Dockerfile Distroless
Base	Imagen completa con un sistema operativo (Ubuntu, Debian, Alpine, etc.)	Imagen mínima, solo incluye el <i>runtime</i> necesario (sin SO ni shell)
Tamaño	Mayor (varios cientos de MB)	Mucho más pequeño (decenas de MB)
Seguridad	Más vulnerable: más paquetes = más superficie de ataque	Más segura: menos componentes, menos CVEs posibles
Herramientas del sistema	Tiene bash, apt, curl, etc.	No tiene shell ni utilidades del sistema
Depuración	Fácil (puede entrar con docker exec -it bash)	Difícil, no se puede acceder directamente, requiere logs y observabilidad
Predictibilidad	Variable según actualizaciones del sistema base	Alta: siempre contiene lo mínimo y controlado
Uso típico	Etapa de desarrollo o entornos con debugging frecuente	Producción o entornos donde prima seguridad y tamaño
Ejemplo de imagen base	FROM openjdk:17-jdk	FROM gcr.io/distroless/java17-debian12

Tabla 2: Elaboración propia.

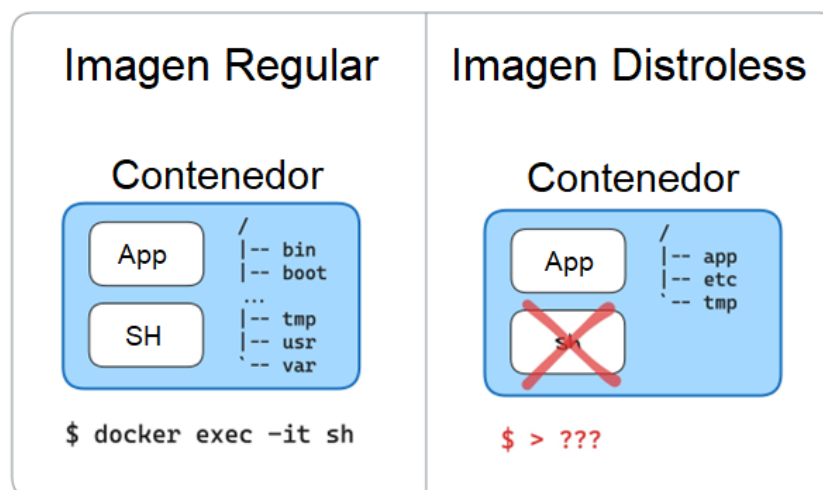


Imagen 8: Elaboración propia.



Atribución-No Comercial-Sin Derivadas

Se permite descargar esta obra y compartirla, siempre y cuando no sea modificado y/o alterado su contenido, ni se comercialice. Universidad Tecnológica Nacional Facultad Regional Córdoba (S/D). Material para la Tecnicatura Universitaria en Programación, modalidad virtual, Córdoba, Argentina.